LearnCpp Chapter 3

Debugging C++ Programs

Field notes, rewritten from scratch • companion to learncpp.com Ch. 3 • July 2026

How these notes work (read this once)

One chapter, one operating system. Same skeleton as the Chapter 2 notes, so your brain never has to ask "what am I looking at?" — it only has to ask "what does this say?"

1. **Meta line** — estimated minutes + what it builds on. Pick sections that fit the time you actually have.
2. **IN ONE BREATH** — the whole section in one sentence. If you remember nothing else, remember this line.
3. **Short chunks** — never a wall of text. Prose, then a box, then code, then prose.
4. **QUICK CHECK** — 30–60 seconds of active recall. *Do not skip it.* Answer out loud or on paper, then check the back. If you miss it, re-read only that section, not the chapter.
5. **Tick the map** — check the box on the progress map. Closure is fuel.

The seven box types (color = meaning, always)

DEFINITION	Precise vocabulary. These words have exact meanings — use them exactly.
KEY IDEA	The mental model. *Why* this thing exists, usually with an analogy.
UNDER THE HOOD	What the machine actually does: memory, compiler, buffers.
GOTCHA	A real bug, always with **When it bites** — the exact phase and trigger.
PRECISION	Two terms people blur together, pulled apart. Say the right one.
BRIDGE	Hooks this idea to something you already own (projects, prior concepts).
QUICK CHECK	Active recall. The learning happens here, not in the reading.

Session protocol (30/60 rhythm)

30 minutes reading ≈ two to three sections, Quick Checks included. **60 minutes doing** = drills from the bank at the back (they cite the sections they test) or a live debugger session on your own code. Whole chapter ≈ two hours of reading spread over as many sittings as you want. If you drift mid-section: name it, tick nothing, come back to the *IN ONE BREATH* line and restart from there.

Progress map

	§	Lesson	min	The one thing to walk away with
☐	3.1	Syntax and semantic errors	8	Compiler enforces grammar; meaning is yours to check.
☐	3.2	The debugging process	8	Five steps; root cause before repair.
☐	3.3	A strategy for debugging	10	You can't fix what you can't reproduce.
☐	3.4	Basic debugging tactics	12	Trace with `std::cerr`; know why prints are a crutch.
☐	3.5	More debugging tactics	12	`#ifdef` toggles; logs beat console spam.
☐	3.6	Debugger: stepping	15	Step into / over / out = three zoom levels.
☐	3.7	Debugger: run & breakpoints	12	Breakpoint = permanent tollbooth; run-to-cursor = one-shot.
☐	3.8	Debugger: watching variables	10	State over time, without a single print.
☐	3.9	Debugger: the call stack	12	Section 2.5's frame diagram, rendered live.
☐	3.10	Finding issues early	12	The cheapest bug is the one that can't exist.

After 3.10: the Big Picture page (debugging loop + symptom triage), the one-page cheat sheet, and the 15-drill bank.

3.1 Syntax and semantic errors

TIME 8 min • BUILDS ON 2.7 (the compile pipeline)

> **IN ONE BREATH** *The compiler enforces grammar (syntax); it cannot enforce meaning (semantics) — so an entire class of errors is yours alone to find.*

DEFINITION • The two species of error

A **syntax error** is a statement that violates the grammar of C++ — a missing semicolon, an unmatched brace, a keyword misspelled. The compiler catches every one at compile time, because it literally cannot translate what it cannot parse. A **semantic error** is grammatically valid code whose *meaning* is not what you intended. Some semantic errors break the language's own rules (initializing a `double` from a string literal) and are also caught at compile time. The rest — **logic errors** — compile perfectly and misbehave only when run: wrong output, a crash, or undefined behavior.

Where each species lives. Map this onto the 2.7 pipeline: syntax and type-rule violations die in the compiler. Everything that survives to a linked executable can only fail at runtime — and runtime failure is exactly where the compiler's protection ends and this chapter begins.

PRECISION • Compile error vs. runtime error vs. UB

A **compile error** means no program was ever produced. A **runtime error** means the program exists and *observably* misbehaves while running — it crashes, or prints the wrong thing. **Undefined behavior** is sharper than either: the standard places *no requirement* on what happens. UB may crash, may print garbage, may look perfectly correct today and differ tomorrow between build types. Never say "runtime error" when you mean UB — a runtime error is at least reliably observable; UB owes you nothing.

UNDER THE HOOD • Why the compiler can't catch logic

The compiler checks your code's *form* against the grammar and the type rules. It has no model of your *intent*. `return x - y;` inside a function named `add` is flawless C++ — every token legal, every type correct. Only you know it should have been `+`. No future compiler flag fixes this: intent isn't in the source.

GOTCHA • "It compiles" is not evidence

A clean build proves only that your program is grammatical and type-correct. **When it bites:** runtime, often silently — a value is simply wrong and nothing crashes to alert you. Trigger: treating zero compiler diagnostics as a correctness signal and skipping the run-and-verify step.

QUICK CHECK • 3.1 • answers at the back

Q1. Classify: `int x = 5` (no semicolon) vs. an `add` function that returns `x - y`. **Q2.** Is integer division by zero a syntax error, a semantic error, or something with a sharper name? **Q3.** Can a program with zero compiler diagnostics still contain UB?

3.2 The debugging process

TIME 8 min • BUILDS ON 3.1

> **IN ONE BREATH** *Five steps — find the root cause, understand it, determine a fix, repair, retest — and the first step eats most of the clock.*

DEFINITION • The five steps

Once a problem is observed, debugging is: **(1) find the root cause** — locate the actual code producing the misbehavior, not the place where you noticed it; **(2) understand the problem** — know *why* that code misbehaves before touching it; **(3) determine a fix**; **(4) repair the issue**; **(5) retest** — confirm the problem is gone *and* that the fix broke nothing else. In real work, step 1 dominates: finding is expensive, fixing is usually trivial once found.

KEY IDEA • The mechanic who replaces parts

A bad mechanic hears a rattle and starts swapping components until the noise stops — expensive, slow, and the rattle usually returns. A good mechanic reproduces the rattle, traces it to the loose bracket, understands why it loosened, and *then* reaches for a tool. Changing code before locating the root cause is part-swapping: even when the symptom disappears, you don't know what you fixed — or what you broke.

PRECISION • Symptom vs. root cause

The **symptom** is the observable misbehavior — the wrong number on screen, the crash. The **root cause** is the code fact producing it — the wrong operator, the discarded return value. One root cause can wear many symptoms; fixing a symptom leaves the cause free to resurface in new costumes. Diagnose causes; observe symptoms.

GOTCHA • Patching the symptom

Output is one too high, so you subtract 1 just before printing. **When it bites:** later — a different call path hits the same root cause and shows a brand-new symptom, while the paths that were *correct* are now off by one in the other direction. Trigger: editing where the error is *visible* instead of where it is *produced*.

QUICK CHECK • 3.2 • answers at the back

Q1. Put the five steps in order from memory. **Q2.** Retest guards against two distinct failures — name both. **Q3.** A teammate says: "Fixed it — I adjusted the final total by +1 before printing." Which steps got skipped?

3.3 A strategy for debugging

TIME 10 min • BUILDS ON 3.2

> **IN ONE BREATH** *You cannot diagnose what you cannot observe: reproduce the problem first, then let the symptom's position rule out code and bisect what remains.*

DEFINITION • Reproduction

Reproducing a problem means making it appear on demand, consistently. **Reproduction steps** are a precise, ordered list of actions and inputs that trigger the issue with high predictability — precise enough that a stranger could follow them. A 100% repro rate is ideal, but partial is workable: an issue that appears half the time simply takes twice as many runs to interrogate, because the silent runs contribute nothing.

KEY IDEA • The search region shrinks twice

First cut: the bug must live in code that executed *between program start and the first bad symptom you can observe* — everything after that point is ruled out instantly. Second cut: program structure. If a pipeline runs read → validate → compute → print, the validation log looks right, and the printed value is wrong, the bug lives in compute or print — you just eliminated half the program without reading a line of it. Each observation is a bisection step on the execution timeline.

BRIDGE • Constraint-first thinking, inverted

In design, the constraints determine the solution — your rule. Debugging is the same muscle pointed backward: each symptom is a *constraint on where the bug can live*. The discipline is identical: before acting, ask "what does this observation make impossible?" The engineer who asks that repeatedly converges; the one who doesn't wanders.

QUICK CHECK • 3.3 • answers at the back

Q1. Why is reproduction the first move rather than reading code? **Q2.** A bug reproduces 25% of the time — what does that change about the diagnosis, and what doesn't it change? **Q3.** In get → sort → print, the names come out in reverse order. Which function do you suspect first, and what did the symptom's *nature* eliminate?

3.4 Basic debugging tactics

TIME 12 min • BUILDS ON 3.3

> **IN ONE BREATH** *Comment code out, print which functions run, print what values flow — with* `std::cerr`*, not* `std::cout` *— and know why this whole toolkit is a crutch.*

Three manual tactics. (1) Comment out the suspect region and rerun: if the symptom changes, the region matters; if nothing changes, look elsewhere. **(2) Print function names** at the top of each function to see the *call flow* — functions called too often, too rarely, or never. **(3) Print values** at checkpoints to see *where* a good value turns bad.

UNDER THE HOOD • Buffered vs. unbuffered output

`std::cout` is **buffered**: characters are parked in a memory buffer and pushed to the terminal later, in batches — system calls are expensive, batching amortizes them. `std::cerr` is **unbuffered**: every insertion goes out *immediately*. The trade is speed vs. punctuality. If the program dies between buffering and flushing, buffered text is *lost* — your trace now lies about how far execution got. Errors and traces need punctuality, which is exactly why `cerr` exists.

PRECISION • cout vs. cerr vs. clog

`std::cout`: buffered, standard output stream, for the *program's* output. `std::cerr`: unbuffered, standard error stream, for errors and debug traces that must appear *now*. `std::clog`: buffered, also the error stream, intended for logging. And they are separately redirectable: `./prog > out.txt` captures `cout` into the file while `cerr` stays on your terminal — traces and output never mingle.

GOTCHA • Tracing with cout

You sprinkle `std::cout` traces, the program crashes, and the last few traces never appear. **When it bites:** runtime, at any abnormal termination — unflushed buffer contents vanish, execution *appears* to have stopped earlier than it did, and you spend an hour searching code that already ran fine. Trigger: any crash between buffering and flush. Trace with `cerr`.

KEY IDEA • Why prints are a crutch

Four standing criticisms: debug prints **clutter the code**; they **clutter the output**; adding and removing them means *editing the program you're diagnosing*, which can introduce new bugs; and once removed they're gone — **not reusable** next time. Keep the technique for quick checks, but everything prints do, the debugger (3.6–3.9) does without touching a single line of source.

QUICK CHECK • 3.4 • answers at the back

Q1. One-word property that makes `cerr` the right trace stream? **Q2.** A `cout`-based trace is missing its last two lines after a crash — did those statements execute? **Q3.** Name two of the four criticisms of print debugging.

3.5 More debugging tactics

TIME 12 min • BUILDS ON 3.4 + 2.10 (preprocessor)

> **IN ONE BREATH** *Wrap debug prints so the preprocessor can strip them with one toggle, or better, send them to a timestamped log file instead of the console.*

Conditionalized tracing. Instead of deleting traces when you're done, gate them:

```
#define ENABLE_DEBUG // comment this line out to silence all tracing

#ifdef ENABLE_DEBUG
std::cerr << "getUserInput() called\n";
#endif
```

One line now flips every trace in the file — and the stripped statements cost *nothing* at runtime, because the compiler never saw them. The 2.10 text machine removed them before compilation began.

DEFINITION • Log, logging, log file

A **log** is a sequential record of events, usually timestamped. Writing to it is **logging**; when it lives on disk it's a **log file**. Two structural advantages over console prints: debug output is *physically separated* from program output, and the file survives the run — a user can send you the log from a failure you

never witnessed. C++ ships `std::clog` (which defaults to the error stream), but real projects use a dedicated logging library — learncpp demonstrates with *plog*: initialize once, then write leveled entries anywhere.

GOTCHA • The silently disabled trace

The preprocessor runs per translation unit, top-down (2.8, 2.10). If `ENABLE_DEBUG` is misspelled, or defined *below* the `#ifdef` that tests it, or simply never defined in one particular file — that tracing silently compiles out. **When it bites:** at runtime, as an *absence*: expected trace lines just don't appear, and the compiler said nothing because nothing is illegal. Trigger: multi-file programs where each .cpp must define the toggle itself.

BRIDGE • Two hooks

First: this is 2.10 doing production work — conditional compilation is the preprocessor's day job, not a parlor trick. Second, your endgame: a matching engine in production cannot pause for a debugger and cannot afford console I/O inside a latency budget — *post-mortem logs are the debugger there*. Every serious trading system is diagnosed from its logs. Learn the toy version now; the discipline transfers whole.

QUICK CHECK • 3.5 • answers at the back

Q1. Which machine removes the gated statements, and at what phase relative to compilation? **Q2.** Why is a misspelled `ENABLE_DEBUG` not a compile error? **Q3.** Two advantages of a log file over console prints?

3.6 Using an integrated debugger: stepping

TIME 15 min • BUILDS ON 3.4, 3.5

> **IN ONE BREATH** *A debugger pauses a live program under your control; step into descends into calls, step over treats them as atomic, step out finishes the current function and ascends.*

DEFINITION • Debugger

A **debugger** is a program that controls how *another* program executes and examines its state while it runs — pause anywhere, advance one statement at a time, inspect any variable, all *without modifying the source*. An **integrated** debugger is one built into your editor. Everything 3.4 and 3.5 did by editing code, the debugger does from outside it.

Precondition: a debug build. The debugger's powers depend on how the executable was built.

UNDER THE HOOD • Debug vs. release build

A **debug build** turns optimizations off and embeds *debugging symbols*: the mapping from machine code back to file, line, and variable names (your toolchain: `-g -O0`). A **release build** optimizes (`-O2`): the optimizer folds constants, deletes variables it can prove unnecessary, merges and reorders lines. The neat line-by-line correspondence between source and machine code *dissolves* — which is why

stepping through a release build feels haunted: the arrow jumps, and variables report "optimized out." Debug what the machine is actually running only when you must; debug the debug build otherwise.

The three stepping verbs. Step into: execute the next statement; if it contains a function call, *descend* — pause at the top of the callee. **Step over**: execute the next statement; if it contains a call, run the entire callee and pause *after* it returns — the call is treated as one atomic unit. **Step out**: run the remainder of the current function and pause back in the caller.

KEY IDEA • Zoom levels

Think of a map. *Step into* zooms down to street level inside the call. *Step over* treats the whole city as a single dot and keeps driving. *Step out* zooms back up to the highway you came from. You already trust the callee? Step over. Suspect it? Step into. Realize mid-descent you're in the wrong place? Step out.

GOTCHA • Two classic stepping traps

(1) Step into on `std::cout << value` descends into the standard library's `operator<<` — a foreign file opens and you're lost in code you didn't write. Recover: step out, close the file. **(2)** Because `cout` is buffered (3.4), a value you just printed may not *display* while stepping. Temporarily add `std::cout << std::unitbuf;` at the top of `main` to force a flush after every insertion — and remove it afterward; it exists only for the session. **When it bites:** mid-session — you conclude you're lost, or that output "didn't happen," when both are artifacts of the tools.

BRIDGE • Your toolchain

In your `new_cpp` scaffold, `CMAKE_BUILD_TYPE=Debug` is what hands the compiler `-g -O0`. And the command line came first: *gdb* is the ancestral debugger, and its verbs map one-to-one — `step` = step into, `next` = step over, `finish` = step out. IDE buttons are a GUI over the same three ideas; learn the ideas and every debugger is yours.

QUICK CHECK • 3.6 • answers at the back

Q1. The arrow sits on `int z{ add(x, y) };`. Where does one press of step into leave you? One press of step over? **Q2.** Why can't the debugger reliably show variables in a release build? **Q3.** gdb's `next` corresponds to which verb?

3.7 Using an integrated debugger: running and breakpoints

TIME 12 min • BUILDS ON 3.6

> **IN ONE BREATH** *Stepping is walking; run to cursor sprints once to a disposable finish line; a breakpoint is a permanent tollbooth execution cannot pass without stopping.*

DEFINITION • The running vocabulary

Run to cursor: execute at full speed until reaching the line under your cursor, then pause — a one-shot, disposable target. **Continue**: resume full-speed execution until the next breakpoint or the program ends. **Start**: launch the program fresh from the beginning, with breakpoints active. A **breakpoint** is a marker on a line telling the debugger: pause *every time* execution reaches here.

KEY IDEA • Breakpoint beats run-to-cursor when the line is popular

A loop body, or a function called from five sites: run-to-cursor pauses on the *first* arrival only, and needs your cursor babysat every restart. A breakpoint pauses on *every* pass, survives restarts, and works when you don't even know which path reaches the line. Alternate the modes: *run* to the interesting neighborhood, then *step* through it statement by statement.

PRECISION • Stepping vs. running

Stepping advances at statement granularity — you inspect after every single move. **Running** is free execution until an *event*: a breakpoint, the cursor line, or program end. A debug session is a rhythm of the two; all-stepping is unbearably slow, all-running observes nothing.

One more power: set next statement. You can drag execution to a different line — skipping code, or jumping back to re-watch a statement run. Useful; dangerous.

GOTCHA • Jumps lie about state

Skipping forward over `init()` means everything `init` would have produced *never existed*. Jumping backward does not rewind: changes already made to variables persist — there is no undo, only a program counter teleport. **When it bites:** immediately downstream — reads of never-produced state, "impossible" values — and you are now debugging a program state that *cannot occur naturally*, chasing ghosts of your own creation. Trigger: any jump that crosses state-producing code.

QUICK CHECK • 3.7 • answers at the back

Q1. Breakpoint on the single statement inside a 5-iteration loop: how many pauses? Run-to-cursor to the same line: how many? **Q2.** What does jumping *backward* with set-next-statement fail to undo? **Q3.** "Resume until something interesting happens" — which command?

3.8 Using an integrated debugger: watching variables

TIME 10 min • BUILDS ON 3.7

> **IN ONE BREATH** *A watch pins a variable or expression to a window that updates at every pause — state over time, without a single print.*

DEFINITION • Watching

Watching a variable means inspecting its value while the program executes under the debugger, across time. The **watch window** holds variables and expressions you *explicitly* add; it re-evaluates them at every pause and typically highlights values that changed since the last stop. The **locals**

window shows *all* variables of the current scope automatically — zero setup, refreshed at every pause.

KEY IDEA • Declarative value-tracing

The 3.4 tactic was imperative: "print `x` here, and here, and here." A watch is declarative: "*always* show me `x`" — the debugger handles when and where. Watches also take *expressions*: `x * y`, `price - fee` — evaluated fresh at each pause. One caution: an expression with side effects (say, a call that increments a counter) *mutates the program you're observing* every time the window refreshes. Watch pure expressions.

BRIDGE • Stored vs. computed, made visible

The design tension from your grade-ledger exercise renders literally on screen: the **locals window is stored state** — what actually occupies the frame; an **expression watch is computed state** — derived on demand, existing nowhere in memory. The debugger doesn't resolve the tension, but it makes the distinction impossible to ignore.

QUICK CHECK • 3.8 • answers at the back

Q1. Locals window vs. watch window — which is automatic, and what does the other add? **Q2.** You suspect `total` drifts wrong somewhere inside one 30-line function: which watch plus which stepping verb exposes the exact line? **Q3.** Why is watching `nextId()` risky if it increments a counter?

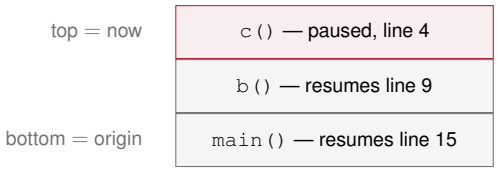
3.9 Using an integrated debugger: the call stack

TIME 12 min • BUILDS ON 3.8 + 2.5 (stack frames)

IN ONE BREATH *The call stack window lists every active function from current (top) down to main (bottom) — section 2.5's frame diagram, rendered live.*

DEFINITION • The call stack

The **call stack** is the list of all *active* functions — called, not yet returned — in the order they were called to reach the current point of execution. The **call stack window** displays it: the current function on top, `main` at the bottom, and each entry annotated with the line that function will resume at. Double-click any entry to view that frame's source position and its locals.



PRECISION • Active vs. called

The window shows functions that have *started and not yet returned*. A function that ran and returned is **popped** — absent from the window, its frame gone. "This function was called earlier" does *not*

imply "this function is on the stack now." Popped, not wiped — and popped means invisible here.

KEY IDEA • The stack answers "how did I get here?"

A breakpoint fires deep inside a helper that five different callers use. The values tell you *what* state you're in; the call stack tells you *which path produced it* — the breadcrumb trail from `main` to this exact pause. For any function with multiple callers, the stack window is the difference between diagnosing and guessing.

BRIDGE • Receipts, and what's coming

2.5 asked you to trust that frames stack up and pop off — this window is the receipt: the diagram was real all along. And ahead of you: the maze solver's recursion will put the *same function* on the stack N times at once — the window becomes your recursion depth gauge, one entry per unfinished descent.

QUICK CHECK • 3.9 • answers at the back

Q1. `a()` calls `b()`, `b()` calls `c()`; paused inside `c`. Write the window top to bottom. **Q2.** `c` returns; `b` then calls `d()`; paused inside `d`. Write it again. **Q3.** Why is `c` absent in Q2 — exact vocabulary?

3.10 Finding issues before they become problems

TIME 12 min • BUILDS ON everything above

> **IN ONE BREATH** *The cheapest bug is the one that can't exist: shrink functions, maximize warnings, test hostile inputs, add defensive checks, and let static analyzers read your code.*

KEY IDEA • The cost curve

A defect costs least at the keyboard, more at compile time, more in testing, and most in production — each stage it survives multiplies the price of finding it. Every practice in this section moves detection *leftward* along that curve. In your target domain the right end of the curve is priced in money per millisecond: production trading systems make "detected late" unusually expensive.

Don't make the error. The unglamorous half of debugging is prevention: follow best practices; don't program when tired or frustrated; know where the common pitfalls live; keep functions short and single-purpose; prefer the standard library to hand-rolled code; comment *why*, not *what*; and test each small unit as you write it rather than integrating a mystery at the end.

DEFINITION • Three prevention tools

Refactoring: restructuring code *without changing its behavior* — typically splitting an overgrown function — so each piece is testable and readable in isolation. **Defensive programming**: anticipating misuse and bad input, and detecting them *loudly* at the point of entry instead of letting them corrupt state downstream. **Static analysis**: tools that inspect source *without running it* and flag suspect patterns. Your compiler is the first static analyzer you own — raise its voice: `-Wall -Wextra` turns on warnings, and warnings are free bug reports. Dedicated analyzers go further down the same road.

GOTCHA • Testing only the happy path

The function was "tested" — with friendly values that mirror your own assumptions. **When it bites:** the first hostile input — zero, negative, enormous, empty — often long after you've moved on, in exactly the code you were most confident about. Trigger: writing tests that confirm your mental model instead of attacking it.

BRIDGE • You already built this

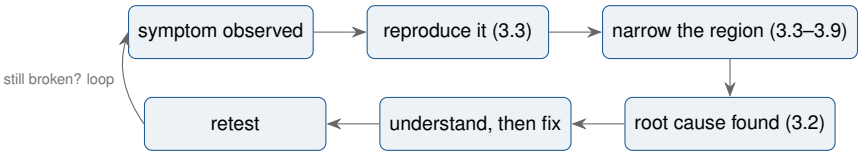
Your order validator *is* defensive programming: the library boundary converting bad input into loud, typed results (`enum class`, not silent corruption) before anything downstream can trust garbage. This section is also where "make it *right*" lives in your make-it-work / make-it-right / make-it-fast framework. Assertions — the formal tool for stating "this must be true here" — get full treatment later in the course; queue them, don't chase them now.

QUICK CHECK • 3.10 • answers at the back

Q1. Two reasons short functions carry fewer bugs? **Q2.** What's the relationship between your compiler and a static analyzer? **Q3.** Name the practice: rejecting `price <= 0` at the library boundary with a typed error.

The Big Picture: one loop, one triage table

The whole chapter is this loop. Sections 3.2–3.9 are just labels on its arrows:



Symptom triage: read the observation, pick the tool

Observation	First tool	Why
Wrong value, no crash	breakpoint before the region + watch the variable (3.7, 3.8)	semantic-runtime bug; the watch shows the exact line where good turns bad
Crash, and the `cout` trace is missing lines	retrace with `cerr`, or breakpoint before the crash site (3.4, 3.7)	buffered output died unflushed; the trace is lying about progress
Function never called / called twice	name-print at function tops, or a breakpoint and count the pauses (3.4, 3.7)	this is a *call-flow* question, not a value question
Works in Debug, breaks in Release	suspect UB; rebuild with `-Wall -Wextra` and read every warning (3.1, 3.10)	the optimizer changes UB's costume; correct code survives both builds
Only happens sometimes	stop debugging; write reproduction steps, add logging (3.3, 3.5)	you cannot bisect what you cannot summon

Memorize the pairing, not the rows: **the observation picks the tool.** Never start from a favorite tool and hunt for a use — ask "what did I actually observe?" first, always.

One-page cheat sheet

The five steps (3.2)

- find the root cause
- understand the problem
- determine a fix
- repair
- retest (fix works + nothing else broke)

Streams (3.4)

`cout`	buffered	program output
`cerr`	unbuffered	errors, traces — *now*
`clog`	buffered	logging (error stream)

Debug toggle template (3.5)

```
#define ENABLE_DEBUG // 1 line = master
    switch

#ifdef ENABLE_DEBUG
std::cerr << "trace\n";
#endif
```

Word pairs (say the right one)

- **symptom** (observed) / **root cause** (produced)
- **syntax** (grammar) / **semantic** (meaning)
- **runtime error** (observable) / **UB** (no promises)
- **buffered** (batched later) / **unbuffered** (now)
- **active** (on the stack) / **returned** (popped, absent)

Stepping = zoom (3.6) + gdb names

step into	descend into call	`step`
step over	call is atomic	`next`
step out	finish, ascend	`finish`
breakpoint	pause every hit	`break`
continue	run to next event	`continue`
inspect	value at pause	`print x`
call stack	how did I get here	`backtrace`

Error taxonomy in three lines (3.1)

- syntax → compile error, always caught
- semantic vs. type rules → compile error
- semantic vs. intent (logic, UB) → yours to find

Build types (3.6)

- Debug = `-g -O0`: symbols in, optimizer off
- Release = `-O2`: fast, but source–machine mapping dissolves

Shift-left checklist (3.10)

- short, single-purpose functions
- `-Wall -Wextra`; read every warning
- test hostile inputs, not just friendly ones
- validate at boundaries, fail loudly
- log, don't print

Drill bank — 15 drills, one sitting

Your standard format: one at a time, no peeking ahead. Types: PREDICT (say the outcome before running), SPOT (find the flaw), FIX (repair it), BUILD (write or script it), DESIGN (no code — decomposition only). Run the PREDICT ones afterward and check yourself against reality.

D1. [SPOT • 3.1]

```
int area(int w, int h) { return w + h; }
int x = 5
double d { "hello" };
```

Classify each line: syntax error, semantic error caught at compile time, or semantic error that survives to runtime.

D2. [SPOT • 3.2] Totals print one too high, so a teammate adds `total -= 1;` just before the print. Name what was repaired (in this chapter's vocabulary) and what wasn't; list the skipped steps; predict one concrete way this bites later.

D3. [BUILD • 3.3] Your trade log "sometimes prints the wrong trade count." Write three-line reproduction steps precise enough that a stranger triggers it reliably: exact inputs, exact actions, expected vs. observed.

D4. [PREDICT • 3.4]

```
std::cout << "A";
std::cerr << "B";
std::cout << "C";
crashNow(); // terminates instantly: no buffers flushed
```

What is *guaranteed* to be on the console, what might be missing, and which property decides it?

D5. [FIX • 3.4] A program reads a number with `getUserInput();` the user types 4 but `main`'s variable receives 3. You may add exactly *two* `cerr` lines. Where do they go so one run proves whether the corruption happens inside `getUserInput` or after it returns?

D6. [PREDICT • 3.5]

```
#ifdef ENABLE_DEBUG
std::cerr << "trace\n";
#endif
#define ENABLE_DEBUG
```

Does `trace` print? Name the machine that decides, and the property of how it reads a file.

D7. [BUILD • 3.5] Wrap a `cerr` trace so release builds contain no trace of it. Then answer: does the stripped statement cost anything at runtime, and which phase removed it?

D8. [PREDICT • 3.6]

```
int twice(int v) { return 2 * v; }
int main()
{
    int a{ 3 };
    int b{ twice(a) };
    std::cout << b;
    return 0;
}
```

Paused with the arrow on `int b{ twice(a) };`. One press of step over lands the arrow where? One press of step into instead? And from inside `twice`, one press of step out?

D9. [SPOT • 3.6] Stepping through a Release build: the arrow jumps erratically and a variable reads "optimized out." Name the two build properties a Debug build changes that explain both symptoms.

D10. [PREDICT • 3.7] A breakpoint sits on the single statement inside a loop that runs five times; you press start. How many pauses before the program ends? Same line, no breakpoint, run-to-cursor once: how many? After pause 3 of the first scenario, one press of continue lands you where?

D11. [BUILD • 3.8]

```
int readNumber(int x) { std::cin >> x; return x; }
int main()
{
    int x {};
    readNumber(x);            // line A
    x = x + readNumber(x);    // line B
    std::cout << x;
}
```

Set exactly one breakpoint and one watch to expose why line A accomplishes nothing. State the line, the watched name, and what the watch will show across the call.

D12. [PREDICT • 3.9] `a()` calls `b()`, `b()` calls `c()`; paused inside `c`: write the call-stack window top to bottom. Then `c` returns and `b` calls `d()`; paused inside `d`: write it again. One vocabulary word explains `c`'s absence — which?

D13. [BUILD • 3.6–3.9] Capstone. Your trade log program, zero code edits: script a debugger session — every breakpoint, watch, stack inspection, and stepping verb in order — that verifies the fee calculation of *only the third trade* and nothing else.

D14. [DESIGN • 3.10] Order validator boundary: name three defensive checks, and for each, the *silent downstream failure* it converts into a loud, immediate one.

D15. [DESIGN • 3.2 + 3.10] Write your personal five-line debugging protocol card, future-HFT edition: one line per step of the five-step process, each naming its default tool from this chapter.

Quick Check answers

3.1 — Q1: Missing semicolon = syntax error (compile time); `x - y` in `add` = semantic/logic error — compiles clean, wrong at runtime. **Q2:** Neither — it's grammatical and type-legal; the sharper name is *undefined behavior*. **Q3:** Yes — UB is a runtime-semantics property; the compiler is not required to detect it, and a silent build proves only grammar and types.

3.2 — Q1: Find root cause → understand → determine fix → repair → retest. **Q2:** Confirms the original problem is actually gone, and catches regressions the fix introduced. **Q3:** Steps 1 and 2 — the symptom was edited where it was *visible*; the root cause was never located or understood.

3.3 — Q1: Because diagnosis requires observation: until the bug appears on demand, every hypothesis is untestable and every run wasted. **Q2:** Changes only the cost — roughly four runs per useful observation; the method (reproduce, narrow, bisect) is unchanged. **Q3:** `sort` — the symptom is an *ordering* defect, which eliminates acquisition and printing as first suspects; structure narrowed the region before any code was read.

3.4 — Q1: Unbuffered. **Q2:** Unknown — that's the point: they may have executed and died in the buffer; a `cout` trace can't distinguish "never ran" from "ran, unflushed." **Q3:** Any two of: clutters code; clutters output; adding/removing edits the program under diagnosis and can add bugs; not reusable once removed.

3.5 — Q1: The preprocessor, which runs *before* the compiler ever sees the file. **Q2:** Because an undefined macro is a perfectly legal state — `#ifdef` simply excludes the block; nothing is grammatically wrong, so nothing is reported. **Q3:** Debug output physically separated from program output; the file persists and can be sent to you from failures you never witnessed.

3.6 — Q1: Step into: top of `add`'s body. Step over: `add` runs entirely; the arrow lands after the statement, `z` now initialized. **Q2:** The optimizer folded, deleted, and reordered — the source-to-machine mapping the symbols describe no longer matches reality; variables may not even exist. **Q3:** Step over.

3.7 — Q1: Breakpoint: five pauses, one per pass. Run-to-cursor: one — it's a one-shot target. **Q2:** Variable state — changes already made persist; only the execution position moves. **Q3:** Continue.

3.8 — Q1: Locals is automatic (everything in scope); watch adds hand-picked variables *and expressions*, tracked across pauses. **Q2:** Watch `total`, then step *over* line by line — the press on which the watched value goes wrong names the guilty line. **Q3:** The watch re-evaluates on every pause, so each refresh increments the counter — observation mutates the program.

3.9 — Q1: `c` (top), `b`, `a`, `main` (bottom). **Q2:** `d` (top), `b`, `a`, `main`. **Q3:** `c` returned, so its frame was *popped* — the window shows active frames only.

3.10 — Q1: Fewer interactions to hold in your head at once, and each function is testable in isolation, so defects surface next to their cause. **Q2:** The compiler *is* a static analyzer — the first one you own; `-Wall -Wextra` raises its voice, and dedicated tools extend the same idea. **Q3:** Defensive programming — loud, typed rejection at the boundary instead of silent downstream corruption.